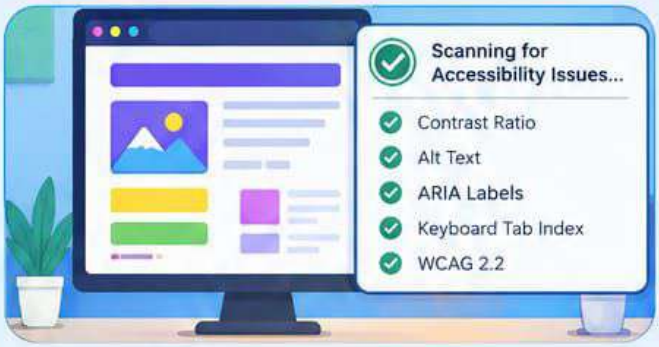
Digital Accessibility & Automated WCAG Contrast / UI Auditor

Make the web accessible | Compliant with WCAG | Build with React + Java + MongoDB

1 What is this project?

An automated website accessibility checker that scans your website or web application for WCAG compliance issues like poor contrast, missing alt text, incorrect ARIA labels, and keyboard navigation problems.



You get:

- Identify accessibility violations
- Generate compliance reports
- Pre-built fix suggestions with code examples
- Improve inclusivity and legal compliance

2 How will your project work?

A step-by-step flow from input to output.

1. **Input URL / Build**
   Developer enters website URL or uploads a frontend build.
2. **DOM Crawling & Parsing**
   System crawls the DOM and analyzes HTML structure.
3. **Accessibility Analysis**
   Checks WCAG rules, contrast ratios, alt text, ARIA labels, and keyboard access.
4. **Compliance Report**
   Generates a detailed report with violations, severity, and exact locations.
5. **Fix Suggestions**
   Provides corrected HTML/CSS/JS examples for each issue.

3 Technology Stack

Modern full-stack architecture for efficient development and scalability.

- **React.js**
  Interactive user interface & auditing dashboard
- **Java (Backend)**
  DOM parsing engine & WCAG compliance logic
- **MongoDB**
  Store audit histories, compliance trends & violation daa
- **HTML/CSS/JavaScript**
  Website structure & styling
- **Selenium**
  Browser automation & DOM inspection
- **WCAG 2.2**
  Accessibility standards (rule validation)

4 Final Output Example

Here's how your dashboard could show the results.

Accessibility Audit Report — Needs improvement

72/100

12	8	35	5
Errors	Warnings	Passed	Fix Suggestions

Detected Violations

Issue	Severity
Missing image alt text	High
Low color contrast	High
Missing form labels	Medium
Incorrect ARIA attribute	Medium

Example Suggested Fix

Before:

```
1  <img src="logo.png">
```

After:

```
1  <img src="logo.png"
       alt="Company logo">
```

Note: This is an illustrative output example, not actual audit results.

Digital Accessibility & WCAG Auditor

Document Type: Technical Specification & System Documentation **Version:** 2.2

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

The **Digital Accessibility & Automated WCAG Contrast / UI Auditor** is an enterprise-grade automated testing suite designed to evaluate web applications for compliance with Web Content Accessibility Guidelines (WCAG 2.2). By integrating headless browser automation with deep DOM parsing logic, the system automatically detects contrast failures, missing alternative text, broken tab indices, and absent ARIA attributes, while outputting precise remediation instructions.

2. PROBLEM STATEMENT & BUSINESS VALUE

Core Challenges Addressed

- **Exclusion of Users:** Millions of users with visual, auditory, motor, or cognitive impairments face barriers on non-compliant web platforms.
- **Legal & Compliance Risks:** Failure to adhere to WCAG standards exposes businesses to ADA lawsuits, regulatory fines, and brand damage.
- **Manual Audit Bottlenecks:** Traditional accessibility reviews require cumbersome manual inspections that stall deployment pipelines.

Delivered Solutions

- **Automated Scanning:** Continuous, rapid assessment of live URLs or frontend code bundles.
- **Actionable Remediation:** Instant generation of before-and-after code snippets for developers.
- **Audit History Tracking:** Long-term storage of compliance metrics and scoring trends via MongoDB.

3. SYSTEM ARCHITECTURE & TECHNOLOGY STACK

The application follows a decoupled full-stack architecture optimized for concurrent parsing operations and responsive auditing dashboards.

Component Tier	Technology	Architectural Responsibility
Frontend Dashboard	React.js	Provides an interactive UI for entering target URLs, rendering audit scores, and viewing detailed violation matrices.
Backend Processing Engine	Java	Handles asynchronous HTTP requests, orchestrates DOM traversal rules, and computes WCAG compliance metrics.
Browser Automation	Selenium	Spins up headless browser instances to render JavaScript-heavy single-page applications (SPAs) and inspect the live DOM tree.
Data Persistence	MongoDB	Stores historical audit logs, violation records, severity metadata, and trend analytics over time.
Validation Standards	WCAG 2.2	Defines the algorithmic ruleset for contrast ratios, semantic elements, and keyboard accessibility.

4. END-TO-END WORKFLOW & EXECUTION PIPELINE

The auditing pipeline processes a target web application through five sequential processing stages:

Step 1: Input Ingestion & Target Specification

The developer submits a target website URL or uploads a frontend build bundle through the React dashboard. The Java backend validates the target accessibility and initializes a scraping session.

Step 2: DOM Crawling & Tree Parsing

Selenium browser automation crawls the Document Object Model (DOM), capturing rendered elements, stylesheets, computed color values, and interactive components across target viewports.

Step 3: WCAG Rule Evaluation & Analysis

The analysis engine scans elements against WCAG 2.2 criteria. It evaluates foreground/background contrast ratios, checks image nodes for valid `alt` attributes, and inspects form elements for associated labels.

Step 4: Compliance Reporting & Severity Scoring

Issues are aggregated into a weighted scoring model (0–100 scale). Violations are categorized by severity levels (High, Medium, Low) with explicit DOM element references.

Step 5: Automated Fix Generation

The system generates context-aware code corrections, showing developers exact before-and-after HTML/CSS snippets for immediate bug patching.

5. SAMPLE REMEDIATION OUTPUT EXAMPLE

Below is an illustration of how the system highlights a missing alternative text violation and recommends the corrected markup:

Violation Detected: Missing image alt text **High Severity**

Before (Non-compliant):

```
<img src="logo.png">
```

After (Suggested Fix):

```
<img src="logo.png" alt="Company Corporate Logo">
```

6. CONCLUSION & FUTURE ROADMAP

By automating the detection of WCAG compliance failures, this tool dramatically reduces audit times and mitigates legal exposure. Future iterations will include automated CI/CD pipeline integration plugins (GitHub Actions / GitLab CI) and AI-driven automated DOM patching.